EduCanvas – GenAI Curriculum Platform

AI-Powered Curriculum Design and Academic Management System using Groq LLaMA

Project Description

EduCanvas is an AI-powered curriculum generation and academic management platform designed to simplify and modernize the process of curriculum planning within educational institutions. The system leverages Generative Artificial Intelligence to assist faculty members in creating structured, semester-wise academic curricula based on specific educational requirements, industry trends, and learning objectives.

The platform enables educators to generate comprehensive curricula by specifying parameters such as degree type, specialization, program duration, weekly learning hours, and industry-focused skill domains. Using the Groq LLaMA Large Language Model, EduCanvas automatically produces detailed curriculum structures that include semester-wise subjects, learning outcomes, skill development pathways, and capstone project recommendations. This significantly reduces the time and effort required for manual curriculum design while ensuring consistency and relevance to current industry standards.

EduCanvas supports a role-based academic ecosystem consisting of faculty members and students. Faculty members can generate, edit, publish, and manage curricula through dedicated dashboards. Generated curricula can be stored in Cloud Firestore, shared with students, and exported as professionally formatted PDF documents using jsPDF and AutoTable libraries. Faculty can also manage student records, educational resources, and curriculum updates through centralized management interfaces.

Students are provided with personalized dashboards where they can browse available curricula, explore detailed course structures, enroll in preferred learning paths, and track their academic progress. The platform maintains enrollment histories and learning records, enabling students to monitor their educational journey efficiently.

The application is built using React.js and Tailwind CSS for the frontend, ensuring a responsive and user-friendly experience across devices. The backend is developed using Node.js and Express.js, providing scalable API services and seamless integration with Groq AI services. Firebase Authentication ensures secure user access through role-based authorization, while Cloud Firestore provides reliable cloud-based storage for users, curricula, enrollments, and analytics data.

EduCanvas addresses the challenges associated with traditional curriculum development by automating curriculum generation, improving academic planning efficiency, reducing administrative workload, and promoting industry-aligned education. Through the integration of Artificial Intelligence, cloud technologies, and modern web development frameworks, EduCanvas delivers a scalable, secure, and intelligent solution for curriculum design, academic management, and educational innovation.

Scenarios

Scenario 1 – Artificial Intelligence Engineering Curriculum

A university faculty member intends to design an Artificial Intelligence Engineering curriculum for undergraduate students. The faculty member enters curriculum requirements such as specialization area, semester count, academic level, and institution details. EduCanvas generates a complete curriculum consisting of semester-wise subjects, course descriptions, learning outcomes, and recommended academic pathways. The generated curriculum can be reviewed, modified, and published for students.

Scenario 2 – Professional Certification Program

A training institute plans to launch a Full Stack Development certification program. Using EduCanvas, instructors generate an industry-oriented curriculum covering frontend development, backend technologies, databases, cloud deployment, and project-based learning. The curriculum is organized into structured modules and exported as a professional syllabus document. Additionally, faculty members can customize course duration, learning outcomes, assessment methods, and project requirements to align with industry standards. The generated curriculum helps institutions quickly introduce certification programs while maintaining consistency and quality across training batches.

Scenario 3 – Student Enrollment and Learning Path Access

Students log into the platform and browse curricula associated with their institution. After reviewing available programs, students enroll in preferred learning paths and track their academic progress through personalized dashboards. The system provides detailed course information, prerequisite recommendations, progress tracking, and completion status for each module. Students can also access learning resources, view upcoming milestones, and receive notifications regarding curriculum updates, assignments, and certification requirements.

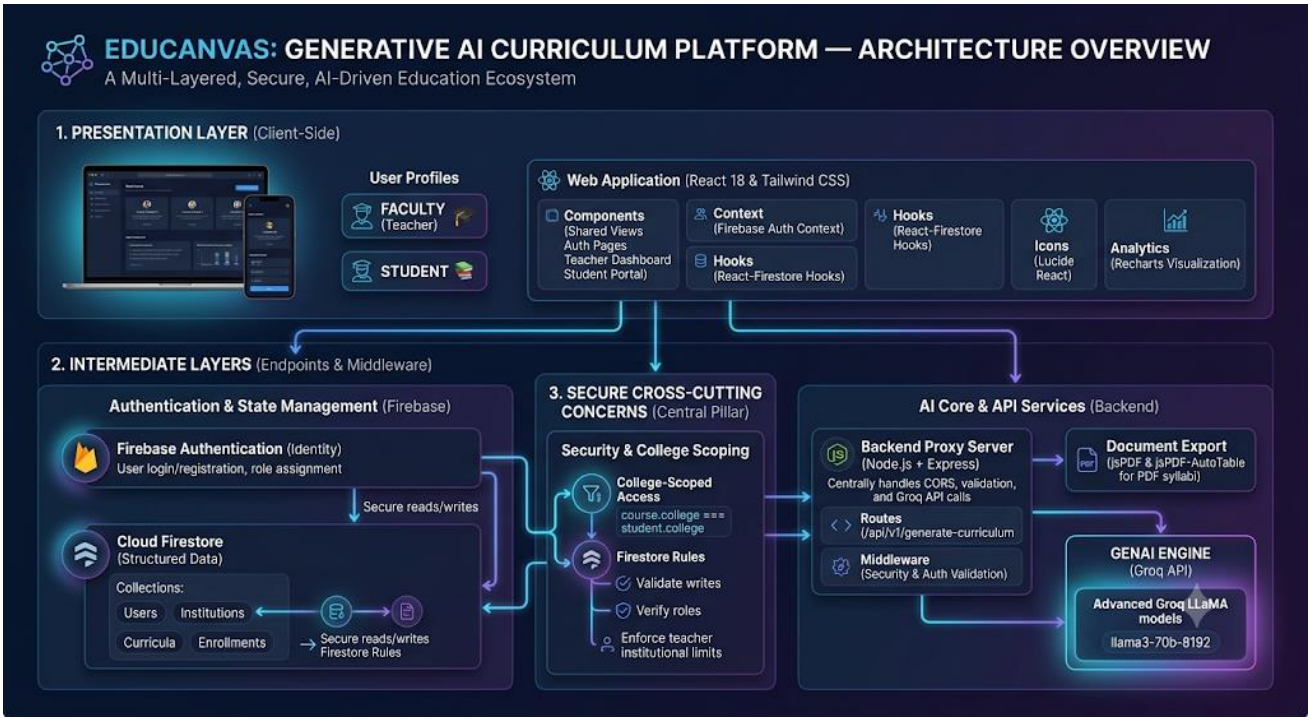
Scenario 4 – Curriculum Analytics and Monitoring

Faculty members use analytics dashboards to monitor curriculum popularity, enrollment statistics, and student participation. These insights assist educators in improving academic offerings and planning future curriculum updates. The platform generates visual reports showing course demand, completion rates, student engagement levels, and performance trends. Administrators can identify highly successful programs, detect areas requiring improvement, and make data-driven decisions to enhance educational effectiveness and institutional outcomes.

Scenario 5 – Faculty Curriculum Collaboration

Multiple faculty members collaborate on curriculum design through a centralized platform. Educators can review existing curricula, suggest modifications, update course content, and share recommendations with department heads. The collaborative workflow ensures curriculum consistency across departments while reducing duplication of effort. Version tracking and approval mechanisms help maintain curriculum quality and compliance with academic standards.

Architecture Overview



EduCanvas follows a modern multi-layered architecture that combines Artificial Intelligence, cloud-based services, secure authentication, and responsive web technologies to provide an intelligent curriculum generation and management platform. The architecture is designed to ensure scalability, security, performance, and ease of use for administrators, faculty members, and students.

1. Presentation Layer

The Presentation Layer serves as the user-facing component of EduCanvas. It is developed using React.js and Tailwind CSS to provide a modern, responsive, and interactive user experience.

Responsibilities

- Displays curriculum creation forms.
- Provides dashboards for administrators, teachers, and students.
- Shows generated curricula and analytics reports.
- Handles user interactions and form validations.
- Supports responsive design for desktop, tablet, and mobile devices.

Technologies Used

- React.js
- Tailwind CSS
- React Router
- Context API

2. Application Layer

The Application Layer acts as the core processing unit of the system. It is built using Node.js and Express.js and manages all business logic and communication between frontend services and backend resources.

Responsibilities

- Processes curriculum generation requests.
- Handles API routing and request validation.
- Manages communication with Groq AI services.
- Processes enrollment and analytics operations.
- Coordinates interactions between Firestore and frontend components.

Technologies Used

- Node.js
- Express.js
- REST APIs
- Middleware Functions

3. Artificial Intelligence Layer

The Artificial Intelligence Layer provides intelligent curriculum generation capabilities using Groq-powered LLaMA models.

Responsibilities

- Generate complete academic curricula.
- Recommend course structures and learning outcomes.
- Create semester-wise academic plans.
- Suggest topics based on skills and educational levels.
- Produce industry-focused learning paths.

Technologies Used

- Groq API
- LLaMA Large Language Models
- Prompt Engineering

4. Authentication Layer

The Authentication Layer ensures secure access to platform resources through Firebase Authentication.

Responsibilities

- User registration.
- User login and logout.
- Password management.
- Session management.

User Roles

Administrator
- Manage institutions and curricula.
- Access analytics dashboards.

Faculty
- Create and manage curricula.
- Monitor student enrollment.

Student
- Browse available programs.
- Enroll in learning paths.
- Track academic progress.

Technologies Used
- Firebase Authentication
- JWT Tokens
- Role-Based Access Control (RBAC)

5. Database Layer

The Database Layer uses Firebase Cloud Firestore to store and manage platform data.

Stored Information
- User Profiles
- Curriculum Records
- Enrollment Data
- Institution Information
- Analytics Data
- Activity Logs

6. Export Layer

The Export Layer enables professional document generation and syllabus distribution.

Responsibilities
- Generate PDF curriculum documents.

- Create formatted academic reports.
- Export semester-wise syllabi.
- Produce printable curriculum templates.

Technologies Used

- jsPDF
- jspdf-autotable

Features

- Professional formatting.
- Course tables and semester layouts.
- Institution branding support.
- Downloadable and shareable PDFs.

Data Flow in EduCanvas

1. User logs into the system using Firebase Authentication.
2. User enters curriculum requirements through the React interface.
3. Express API receives the request.
4. Groq LLaMA model generates curriculum recommendations.
5. Generated data is stored in Firestore.
6. Curriculum is displayed on the dashboard.
7. Users can download the curriculum as a PDF using jsPDF and AutoTable.
8. Analytics data is collected and displayed to faculty and administrators.
9.

Core Technologies

Frontend Technologies

- React.js
- Vite
- Tailwind CSS
- Lucide React
- Recharts

Backend Technologies

- Node.js
- Express.js

Database Technologies

- Firebase Firestore

Authentication Technologies

- Firebase Authentication

Artificial Intelligence Technologies

- Groq API
- LLaMA Large Language Model

Document Generation Technologies

- jsPDF
- jsPDF AutoTable

Pre-Requisites Software

Requirements

- Node.js Version 18+
- npm Package Manager
- Firebase Project
- Groq API Key
- Visual Studio Code
- Google Chrome Browser

Knowledge Requirements

- JavaScript
- React.js
- Node.js
- Firebase
- REST APIs
- Database Fundamentals

Hardware Requirements

- Intel i5 / AMD Ryzen 5 Processor
- Minimum 8 GB RAM
- 10 GB Storage

Project Workflow

Phase 1: Environment Setup and Configuration

Establish development environment and configure external services.

Activity 1.1: Project Initialization

- Create frontend and backend project structures.
- Configure package dependencies.
- Establish development environment.

Activity 1.2: Firebase Configuration

- Create Firebase project.
- Enable Authentication.
- Configure Firestore Database.
- Define security rules.

Activity 1.3: Groq API Configuration

- Generate API key.
- Configure environment variables.
- Test AI connectivity.

Phase 2: Backend Development

Build server-side services and API infrastructure.

Activity 2.1: Express Server Setup

- Configure Express application.
- Create API routes.
- Implement middleware.

Activity 2.2: Authentication Services

- User registration.
- User login.
- Session validation.
- Role verification.

Activity 2.3: Curriculum Generation Service

- Prompt construction.
- AI communication.
- Response parsing.
- Curriculum formatting.

Activity 2.4: Firestore Integration

- Store curriculum data.
- Manage user records.
- Handle enrollments.

Activity 2.5: PDF Generation Module

- Generate curriculum reports.
- Export professional syllabi.
- Create downloadable documents.

Phase 3: Frontend Development

Develop responsive and user-friendly interfaces.

Activity 3.1: Landing Page Design

- Hero section.
- Registration wizard.
- Navigation components.

Activity 3.2: Authentication Interfaces

- Registration page.
- Login page.
- Form validation.

Activity 3.3: Faculty Dashboard

- Curriculum generation interface.
- Curriculum management.
- Analytics visualization.

Activity 3.4: Student Dashboard

- Curriculum browsing.
- Enrollment management.
- Progress tracking.

Phase 4: Integration and Testing

Connect frontend, backend, and cloud services.

Activity 4.1: End-to-End Integration

- API connectivity.
- Database operations.
- Authentication validation.

Activity 4.2: Functional Testing

- Curriculum generation.
- Enrollment workflow.
- PDF export.
- Analytics reporting.

Activity 4.3: Performance Testing

- Response time analysis.
- Database performance.
- Scalability verification.

Phase 5: Deployment and Documentation

Deploy the application and prepare technical documentation.

Activity 5.1: Production Deployment

- Configure hosting.
- Set environment variables.

- Enable monitoring.

Activity 5.2: Documentation Preparation

- Technical documentation.
- User guide.
- API documentation.
- Deployment guide.

Technical Architecture

Frontend Layer

Responsible for user interaction and presentation.

Components:

- Landing Page
- Login Page
- Registration Wizard
- Faculty Dashboard
- Student Dashboard
- Analytics Dashboard

Backend Layer

Responsible for business logic. Components:

- Authentication Service
- Curriculum Service
- Enrollment Service
- Analytics Service

Database Layer

Responsible for data persistence.

Collections:

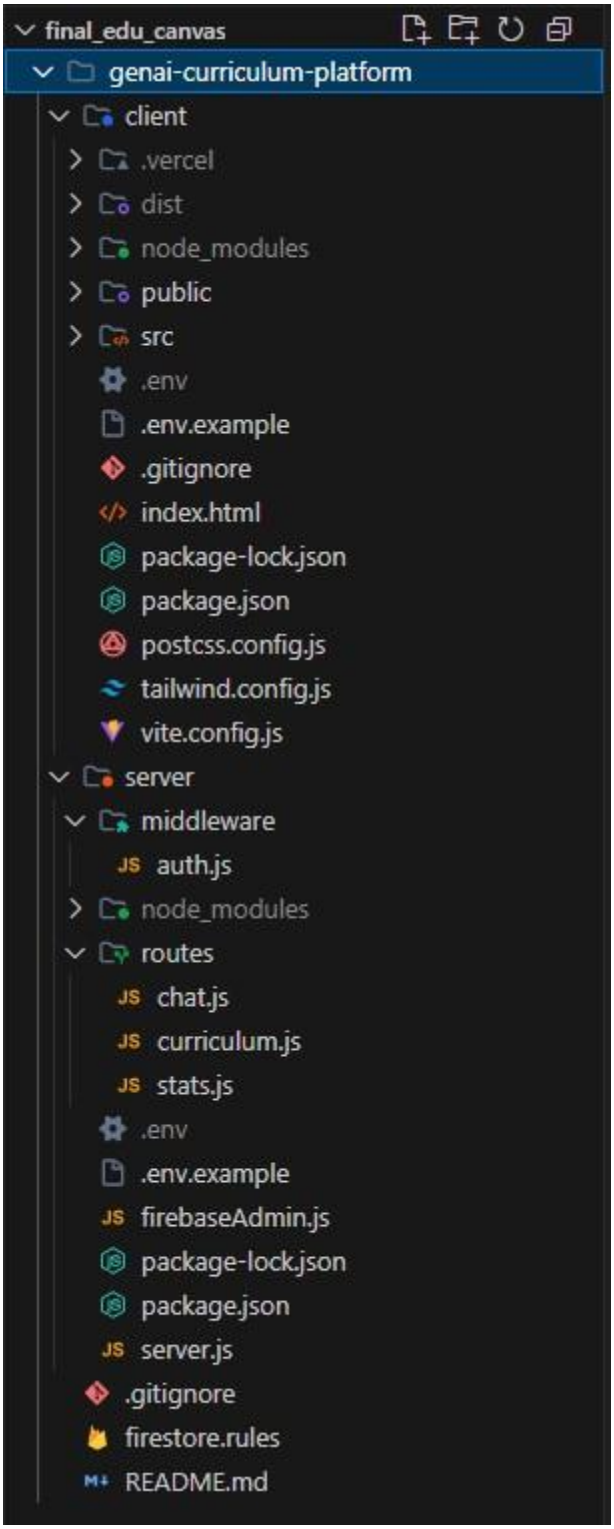
- Users
- Curricula
- Enrollments

- Analytics

AI Layer

Responsible for generating curriculum content.

Project Structure



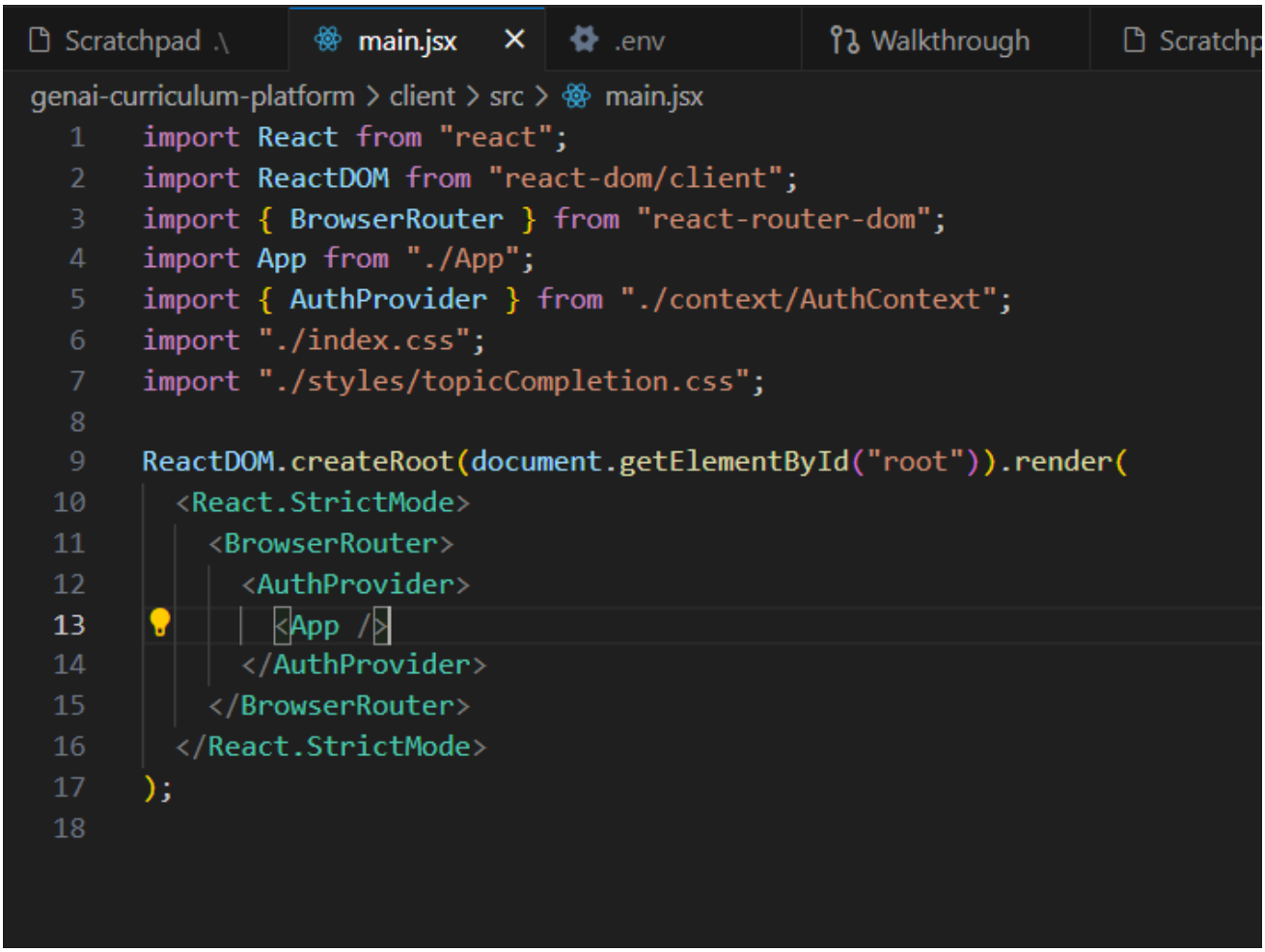
MILESTONE 1

This phase establishes the development environment and configures the services required for the EduCanvas platform. It includes project initialization, Firebase configuration, Firestore setup, and AI integration using Groq.

Activity 1.1: Install Dependencies & Setup Environment

The React application is initialized using Vite and required dependencies are installed.

src/main.jsx



App.jsx

genai-curriculum-platform > client > src > App.jsx > ...

```jsx
import StudentHistory from "./components/student/StudentHistory";

export default function App() {
  return (
    <Routes>
      <Route path="/" element={<CoverPage />} />
      <Route path="/login" element={<Login />} />
      <Route path="/register" element={<Register />} />

      <Route
        path="/teacher"
        element={
          <ProtectedRoute role="teacher">
            <DashboardLayout />
          </ProtectedRoute>
        }
      >
        <Route path="dashboard" element={<TeacherDashboard />} />
        <Route path="generate" element={<GenerateCurriculum />} />
        <Route path="curricula" element={<PostedCurricula />} />
        <Route path="students" element={<ManageStudents />} />
        <Route path="resources" element={<Resources />} />
        <Route path="analytics" element={<Analytics />} />
        <Route path="profile" element={<ProfilePage />} />
        <Route path="history" element={<TeacherHistory />} />
      </Route>

      <Route
        path="/student"
        element={
          <ProtectedRoute role="student">
            <DashboardLayout />
          </ProtectedRoute>
        }
      >
        <Route path="dashboard" element={<StudentDashboard />} />
        <Route path="browse" element={<BrowseCurricula />} />
        <Route path="curriculum/:id" element={<CurriculumDetail />} />
        <Route path="history" element={<StudentHistory />} />
        <Route path="profile" element={<ProfilePage />} />
      </Route>
    </Routes>
  );
}
```

MILESTONE 2: Authentication & Firebase

This phase develops the backend services responsible for curriculum generation, authentication, Firestore operations, and PDF export functionality.

Activity 2.1: Authentication Module

A secure authentication system is implemented using Firebase Authentication.

AuthContext.jsx

genai-curriculum-platform > client > src > context > AuthContext.jsx > ...

```jsx
import { createContext, useContext, useEffect, useState } from "react";
import {
  createUserWithEmailAndPassword,
  onAuthStateChanged,
  signInWithEmailAndPassword,
  signOut,
} from "firebase/auth";
import { doc, getDoc, setDoc, serverTimestamp } from "firebase/firestore";
import { auth, db } from "../firebase/config";

const AuthContext = createContext(null);

export function useAuth() {
  const context = useContext(AuthContext);
  if (!context) throw new Error("useAuth must be used within AuthProvider");
  return context;
}

export function AuthProvider({ children }) {
  const [currentUser, setCurrentUser] = useState(null);
  const [userProfile, setUserProfile] = useState(null);
  const [loading, setLoading] = useState(true);

  async function fetchUserProfile(uid) {
    const snap = await getDoc(doc(db, "users", uid));
    if (snap.exists()) return snap.data();
    return null;
  }

```

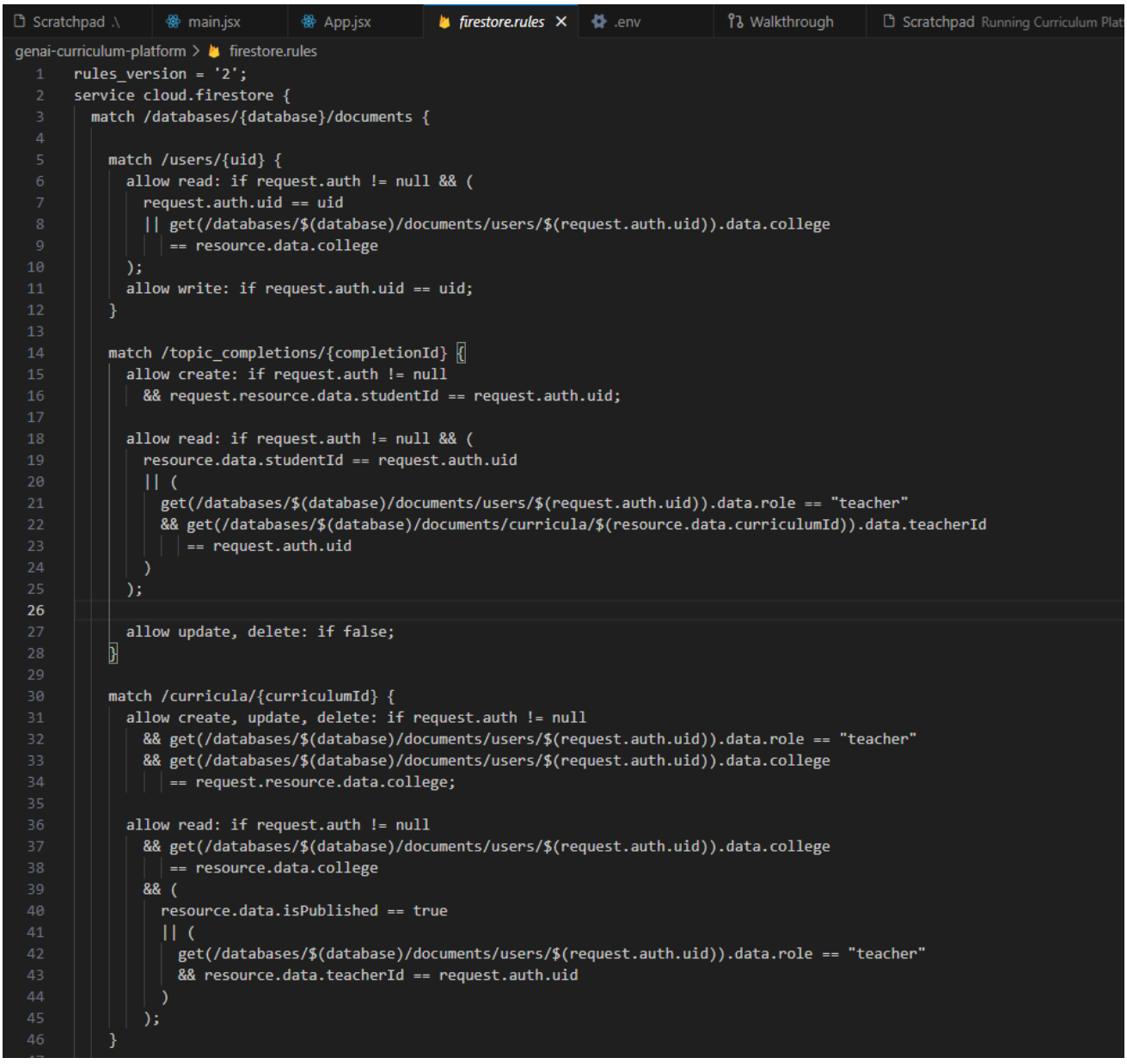
genai-curriculum-platform > client > src > context > AuthContext.jsx > ...

```
19  export function AuthProvider({ children }) {
29
30    async function register({ name, email, password, role, college }) {
31      const cred = await createUserWithEmailAndPassword(auth, email, password);
32      const profile = {
33        name,
34        email,
35        role,
36        college,
37        createdAt: serverTimestamp(),
38      };
39      await setDoc(doc(db, "users", cred.user.uid), profile);
40      setUserProfile(profile);
41      return cred.user;
42    }
43
44    async function login(email, password) {
45      const cred = await signInWithEmailAndPassword(auth, email, password);
46      const profile = await fetchUserProfile(cred.user.uid);
47      setUserProfile(profile);
48      return { user: cred.user, profile };
49    }
50
51    async function logout() {
52      await signOut(auth);
53      setUserProfile(null);
54    }
55
56    useEffect(() => {
57      const unsubscribe = onAuthStateChanged(auth, async (user) => {
58        setCurrentUser(user);
59        if (user) {
60          const profile = await fetchUserProfile(user.uid);
61          setUserProfile(profile);
62        } else {
63          setUserProfile(null);
64        }
65        setLoading(false);
66      });
67      return unsubscribe;
68    }, []);
69
70    const value = {
71      currentUser,
72      userProfile,
73      loading,
74      register,
75      login,
76      logout,
77    };
78
79    return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
80  }
```

Activity 2.2: Firestore Rules

File:

firestore.rules




```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    match /users/{uid} {
      allow read: if request.auth != null && (
        request.auth.uid == uid
        || get(/databases/$(database)/documents/users/$(request.auth.uid)).data.college
           == resource.data.college
      );
      allow write: if request.auth.uid == uid;
    }

    match /topic_completions/{completionId} {
      allow create: if request.auth != null
        && request.resource.data.studentId == request.auth.uid;

      allow read: if request.auth != null && (
        resource.data.studentId == request.auth.uid
        || (
          get(/databases/$(database)/documents/users/$(request.auth.uid)).data.role == "teacher"
          && get(/databases/$(database)/documents/curricula/$(resource.data.curriculumId)).data.teacherId
             == request.auth.uid
        )
      );

      allow update, delete: if false;
    }

    match /curricula/{curriculumId} {
      allow create, update, delete: if request.auth != null
        && get(/databases/$(database)/documents/users/$(request.auth.uid)).data.role == "teacher"
        && get(/databases/$(database)/documents/users/$(request.auth.uid)).data.college
           == request.resource.data.college;

      allow read: if request.auth != null
        && get(/databases/$(database)/documents/users/$(request.auth.uid)).data.college
           == resource.data.college
        && (
          resource.data.isPublished == true
          || (
            get(/databases/$(database)/documents/users/$(request.auth.uid)).data.role == "teacher"
            && resource.data.teacherId == request.auth.uid
          )
        );
    }
```

```
47
48      match /enrollments/{curriculumId}/students/{uid} {
49        allow write: if request.auth.uid == uid;
50        allow read: if request.auth != null;
51      }
52
53      match /teacherHistory/{uid}/entries/{entryId} {
54        allow read, write: if request.auth.uid == uid;
55      }
56
57      match /studentHistory/{uid}/entries/{entryId} {
58        allow read, write: if request.auth.uid == uid;
59      }
60    }
61  }
62
```

MILESTONE 3: Curriculum Generation

This module generates semester-wise curricula using AI.

GenerateCurriculum.jsx

```
async function handleGenerate(e) {
  e.preventDefault();
  setLoading(true);
  setCurriculum(null);
  try {
    const { curriculum: generated } = await generateCurriculum({
      ...form,
      semesters: parseInt(form.semesters, 10),
    });
    setCurriculum(generated);
    await logTeacherAction(
      currentUser.uid,
      "generated",
      "",
      generated.title,
      generated
    );
  } catch (err) {
    showToast(err.response?.data?.error || "Generation failed", "error")
  } finally {
    setLoading(false);
  }
}

async function handlePost() {
  if (!curriculum) return;
  setPosting(true);
  try {
    const docData = {
      title: curriculum.title,
      skill: curriculum.skill,
      level: curriculum.level,
      semesters: parseInt(form.semesters, 10),
      weeklyHours: curriculum.weeklyHours || form.weeklyHours,
      industryFocus: curriculum.industryFocus || form.industryFocus,
      college: userProfile.college,
      teacherId: currentUser.uid,
      teacherName: userProfile.name,
      postedAt: serverTimestamp(),
      isPublished: true,
      status: "active",
      semesterData: curriculum.semesters,
```

Groq API Integration

File:

src/utils/groqAPI.js

```
import axios from "axios";

const API_URL = import.meta.env.VITE_BACKEND_URL || "http://localhost:5000";

export async function generateCurriculum(formData) {
  const response = await axios.post(`${API_URL}/api/curriculum/generate`, formData);
  return response.data;
}

export async function sendChatMessage(message, history, curriculumContext = "") {
  const response = await axios.post(`${API_URL}/api/chat/chat`, {
    message,
    history,
    curriculumContext,
  });
  return response.data;
}

```

Firestore Hooks

File:

src/hooks/

Scratchpad .\ | main.jsx | App.jsx | useFirestore.js | .env | Walkthrough | Scratc

genai-curriculum-platform > client > src > hooks > useFirestore.js > ...

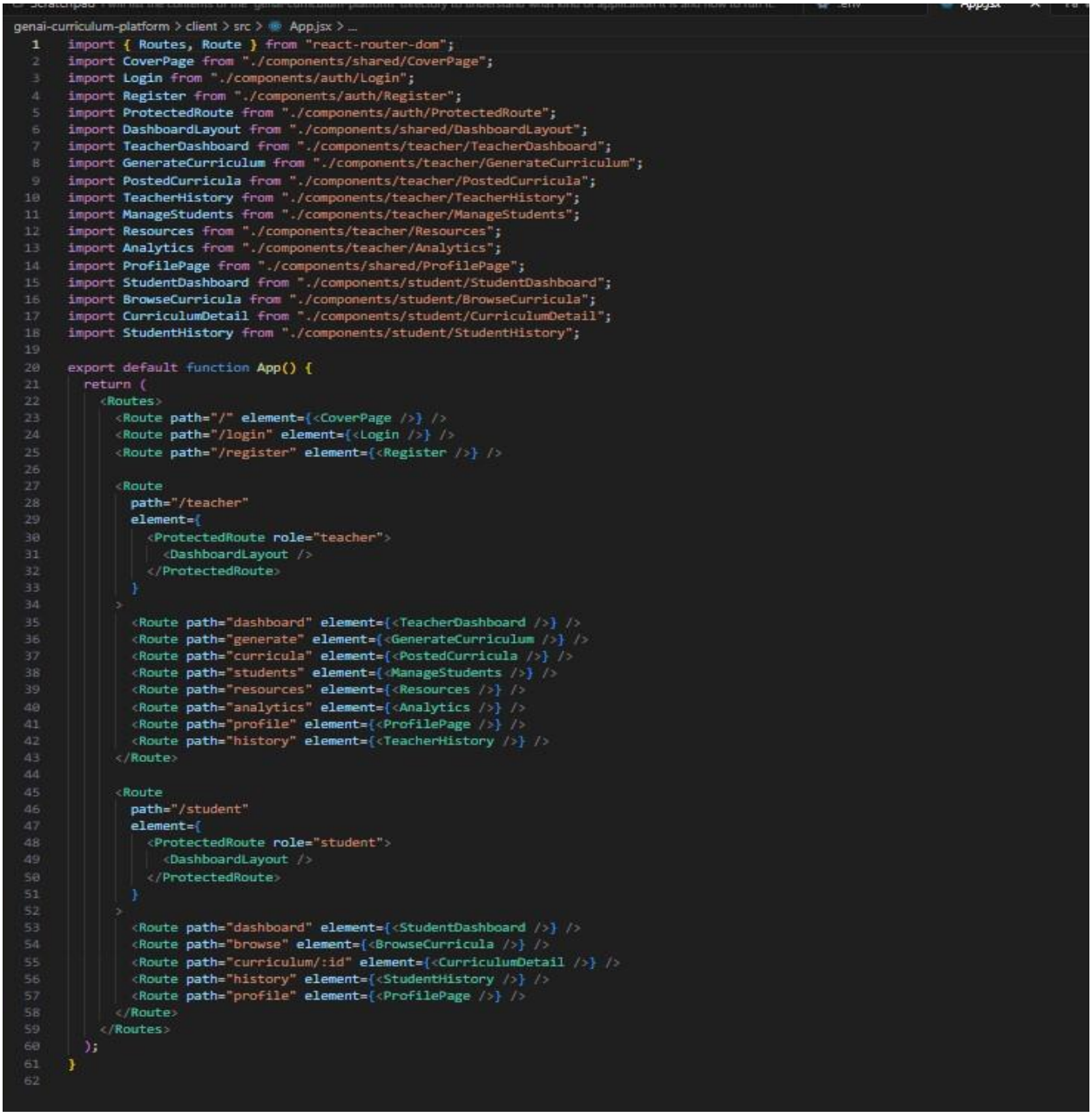
```
import { addDoc, collection, serverTimestamp } from "firebase/firestore";
import { db } from "../firebase/config";

export async function logTeacherAction(uid, action, curriculumId, title, snapshot) {
  await addDoc(collection(db, "teacherHistory", uid, "entries"), {
    curriculumId,
    title,
    action,
    timestamp: serverTimestamp(),
    snapshot,
  });
}

export async function logStudentAction(uid, action, curriculumId, title) {
  await addDoc(collection(db, "studentHistory", uid, "entries"), {
    curriculumId,
    title,
    action,
    timestamp: serverTimestamp(),
  });
}

export async function getEnrollmentCount(curriculumId) {
  const { getDocs, collection: col } = await import("firebase/firestore");
  const snapshot = await getDocs(col(db, "enrollments", curriculumId, "students"));
  return snapshot.size;
}

```

MILESTONE 3: Frontend Development

This phase develops the user interface for faculty and students.

Activity 3.2: Routing Configuration

App.jsx



```
genai-curriculum-platform > client > src > App.jsx > ...
import { Routes, Route } from "react-router-dom";
import CoverPage from "./components/shared/CoverPage";
import Login from "./components/auth/Login";
import Register from "./components/auth/Register";
import ProtectedRoute from "./components/auth/ProtectedRoute";
import DashboardLayout from "./components/shared/DashboardLayout";
import TeacherDashboard from "./components/teacher/TeacherDashboard";
import GenerateCurriculum from "./components/teacher/GenerateCurriculum";
import PostedCurricula from "./components/teacher/PostedCurricula";
import TeacherHistory from "./components/teacher/TeacherHistory";
import ManageStudents from "./components/teacher/ManageStudents";
import Resources from "./components/teacher/Resources";
import Analytics from "./components/teacher/Analytics";
import ProfilePage from "./components/shared/ProfilePage";
import StudentDashboard from "./components/student/StudentDashboard";
import BrowseCurricula from "./components/student/BrowseCurricula";
import CurriculumDetail from "./components/student/CurriculumDetail";
import StudentHistory from "./components/student/StudentHistory";

export default function App() {
  return (
    <Routes>
      <Route path="/" element={<CoverPage />} />
      <Route path="/login" element={<Login />} />
      <Route path="/register" element={<Register />} />

      <Route
        path="/teacher"
        element={
          <ProtectedRoute role="teacher">
            <DashboardLayout />
          </ProtectedRoute>
        }
      >
        <Route path="dashboard" element={<TeacherDashboard />} />
        <Route path="generate" element={<GenerateCurriculum />} />
        <Route path="curricula" element={<PostedCurricula />} />
        <Route path="students" element={<ManageStudents />} />
        <Route path="resources" element={<Resources />} />
        <Route path="analytics" element={<Analytics />} />
        <Route path="profile" element={<ProfilePage />} />
        <Route path="history" element={<TeacherHistory />} />
      </Route>

      <Route
        path="/student"
        element={
          <ProtectedRoute role="student">
            <DashboardLayout />
          </ProtectedRoute>
        }
      >
        <Route path="dashboard" element={<StudentDashboard />} />
        <Route path="browse" element={<BrowseCurricula />} />
        <Route path="curriculum/:id" element={<CurriculumDetail />} />
        <Route path="history" element={<StudentHistory />} />
        <Route path="profile" element={<ProfilePage />} />
      </Route>
    </Routes>
  );
}
```

Purpose:

- Role-based routing
- Protected navigation
- Dashboard access

MILESTONE 4: Backend Development

Express Server Setup

File:

server/index.js

```
require("dotenv").config();
const express = require("express");
const cors = require("cors");
const curriculumRoutes = require("./routes/curriculum");
const chatRoutes = require("./routes/chat");
const statsRoutes = require("./routes/stats");

const app = express();
const PORT = process.env.PORT || 5000;

app.use(cors());
app.use(express.json());

app.use("/api/curriculum", curriculumRoutes);
app.use("/api/chat", chatRoutes);
app.use("/api/stats", statsRoutes);

app.get("/api/health", (_req, res) => {
  res.json({ status: "ok" });
});

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});

```

API Routes

File:

server/routes/*

genai-curriculum-platform > server > routes > curriculum.js > router.post("/generate") callback > prompt

```
require("dotenv").config();
const express = require("express");
const Groq = require("groq-sdk");

const router = express.Router();
const groq = new Groq({ apiKey: process.env.GROQ_API_KEY });

router.post("/generate", async (req, res) => {
  try {
    const { skill, level, semesters, weeklyHours, industryFocus } = req.body;

    if (!skill || !level || !semesters) {
      return res.status(400).json({ success: false, error: "Missing required fields" });
    }

    const prompt = `
You are an expert educational curriculum designer.
Generate a complete, structured academic curriculum in valid JSON format ONLY.
No explanation, no markdown, only raw JSON.

Parameters:
- Skill: ${skill}
- Education Level: ${level}
- Number of Semesters: ${semesters}
- Weekly Study Hours: ${weeklyHours || "Not specified"}
- Industry Focus: ${industryFocus || "General Technology"}

Return this exact JSON structure:
{
  "title": "<Skill> Learning Plan",
  "skill": "${skill}",
  "level": "${level}",
  "weeklyHours": "${weeklyHours || ""}",
  "industryFocus": "${industryFocus || "General Technology"}",
  "totalCourses": <number>,
  "totalCredits": <number>,
  "semesters": [
    {
      "semester": 1,
      "courses": [
        {
          "code": "CS101",
          "name": "<Course Name>",
          "credits": 4,
          "description": "<2-sentence description>",
          "topics": ["Topic 1", "Topic 2", "Topic 3", "Topic 4", "Topic 5"]
        }
      ]
```

```
    }
  ],
  "capstoneProject": "<Capstone project description>"
}

Rules:
- ${semesters} semesters total
- 3 courses per semester
- 4 credits per course
- 5 topics per course
- Topics must be specific, technical, and relevant to ${skill} and ${industryFocus || "General Technology"}
- Courses must progress logically from foundational to advanced
`;

    const completion = await groq.chat.completions.create({
      messages: [{ role: "user", content: prompt }],
      model: "llama-3.3-70b-versatile",
      temperature: 0.4,
      max_tokens: 4000,
    });

    const raw = completion.choices[0].message.content.trim();
    const cleaned = raw.replace(/```json|```/g, "").trim();
    const curriculum = JSON.parse(cleaned);

    res.json({ success: true, curriculum });
  } catch (error) {
    console.error("Curriculum generation error:", error);
    res.status(500).json({
      success: false,
      error: error.message || "Failed to generate curriculum",
    });
  }
});

module.exports = router;
```

Middleware

File:

server/middleware/*

MILESTONE 5: PDF Export

Activity 5.1: PDF Generator

File:

src/utils/pdfExport.js

genai-curriculum-platform > client > src > utils > JS generatePDF.js > ...

```
import jsPDF from "jspdf";
import autoTable from "jspdf-autotable";

export function downloadCurriculumPDF(curriculum, teacherName, college) {
  const doc = new jsPDF();

  doc.setFillColor(59, 130, 246);
  doc.rect(0, 0, 210, 60, "F");
  doc.setTextColor(255, 255, 255);
  doc.setFontSize(24);
  doc.setFont("helvetica", "bold");
  doc.text("EduCanvas", 15, 25);
  doc.setFontSize(11);
  doc.setFont("helvetica", "normal");
  doc.text("AI-Powered Curriculum Platform", 15, 33);

  doc.setTextColor(30, 41, 59);
  doc.setFontSize(20);
  doc.setFont("helvetica", "bold");
  doc.text(curriculum.title, 15, 80);

  doc.setFontSize(11);
  doc.setFont("helvetica", "normal");
  doc.text(`Level: ${curriculum.level}`, 15, 92);
  doc.text(`Weekly Hours: ${curriculum.weeklyHours || "N/A"}`, 15, 100);
  doc.text(`Industry Focus: ${curriculum.industryFocus || "General"}`, 15, 108);
  doc.text(`Teacher: ${teacherName}`, 15, 116);
  doc.text(`College: ${college}`, 15, 124);
  doc.text(`Generated: ${new Date().toLocaleDateString()}`, 15, 132);

  const semesters = curriculum.semesterData || curriculum.semesters || [];
  semesters.forEach((sem) => {
    doc.addPage();
    doc.setFontSize(14);
    doc.setFont("helvetica", "bold");
    doc.setTextColor(59, 130, 246);
    doc.text(`Semester ${sem.semester}`, 15, 20);

    const tableData = sem.courses.map((c) => [
      c.code,
      c.name,
      `${c.credits} Cr`,
      c.topics.join(", "),
    ]);

    autoTable(doc, {
      startY: 28,
      head: [["Code", "Course Name", "Credits", "Topics"]],
      body: tableData,
      styles: { fontSize: 9, cellPadding: 4 },
      headStyles: { fillColor: [59, 130, 246], textColor: 255 },
      columnStyles: { 3: { cellWidth: 80 } },
    });
  });

  doc.addPage();
  doc.setFontSize(14);
  doc.setFont("helvetica", "bold");
  doc.setTextColor(59, 130, 246);
  doc.text("Capstone Project", 15, 20);
  doc.setTextColor(30, 41, 59);
  doc.setFontSize(11);
  doc.setFont("helvetica", "normal");
  const lines = doc.splitTextToSize(curriculum.capstoneProject || "", 180);
  doc.text(lines, 15, 32);

  const filename = `${(curriculum.skill || "curriculum").replace(/\s+/g, "_")}_curriculum.pdf`;
  doc.save(filename);
}
```

MILESTONE 6: UI Components

Activity 6.1: Shared Components

Files:

src/components/shared/

Navbar.jsx

genai-curriculum-platform > client > src > components > shared > Navbar.jsx > ...

```jsx
import { useState, useRef, useEffect } from "react";
import { LogOut, UserCircle, ChevronDown, Share2 } from "lucide-react";
import { useAuth } from "../../context/AuthContext";
import { usernameFromProfile } from "../../utils/profileService";

export default function Navbar() {
  const { logout, currentUser, userProfile } = useAuth();
  const [open, setOpen] = useState(false);
  const menuRef = useRef(null);

  const displayName = usernameFromProfile(userProfile, currentUser?.uid);

  useEffect(() => {
    function handleClick(e) {
      if (menuRef.current && !menuRef.current.contains(e.target)) setOpen(false);
    }
    document.addEventListener("mousedown", handleClick);
    return () => document.removeEventListener("mousedown", handleClick);
  }, []);

  return (
    <header className="flex h-16 shrink-0 items-center justify-between border-b border-slate-200 bg-white px-6">
      <div className="flex items-center gap-2.5">
        <div className="flex h-8 w-8 items-center justify-center rounded-lg bg-gradient-to-br from-cyan-500 to-[#6366f1] text-white">
          <Share2 className="h-4.5 w-4.5" strokeWidth={2.5} />
        </div>
        <span className="text-lg font-bold text-slate-800">
          Edu<span className="text-brand">Canvas</span>
        </span>
      </div>

      <div className="relative" ref={menuRef}>
        <button
          type="button"
          onClick={() => setOpen((v) => !v)}
          className="flex items-center gap-2 rounded-full border border-slate-200 bg-white px-4 py-1.5 text-sm font-medium text-slate
          aria-expanded={open}
          aria-haspopup="true"
        >
          <UserCircle className="h-4 w-4 text-brand" aria-hidden="true" />
          {displayName}
          <ChevronDown className="h-4 w-4 text-slate-400" aria-hidden="true" />
        </button>

        {open && (
          <div className="absolute right-0 z-50 mt-2 w-44 rounded-xl border border-slate-100 bg-white py-1 shadow-lg">
            <button
```

```
                type="button"
                onClick={() => {
                  setOpen(false);
                  logout();
                }}
                className="flex w-full items-center gap-2 px-4 py-2.5 text-sm text-slate-600 hover:bg-slate-50"
              >
                <LogOut className="h-4 w-4" aria-hidden="true" />
                Logout
              </button>
            </div>
          )}
        </div>
      </header>
    );
  }
```

Sidebar.jsx

```jsx
import { NavLink } from "react-router-dom";
import {
  LayoutDashboard,
  Sparkles,
  BookOpen,
  GraduationCap,
  CloudUpload,
  PieChart,
  User,
  Library,
  Clock,
} from "lucide-react";

const teacherLinks = [
  { to: "/teacher/dashboard", label: "Dashboard", icon: LayoutDashboard },
  { to: "/teacher/generate", label: "Generate", icon: Sparkles },
  { to: "/teacher/curricula", label: "My Curricula", icon: BookOpen },
  { to: "/teacher/students", label: "Manage Students", icon: GraduationCap },
  { to: "/teacher/resources", label: "Resources", icon: CloudUpload },
  { to: "/teacher/analytics", label: "Analytics", icon: PieChart },
  { to: "/teacher/profile", label: "Profile", icon: User },
];

const studentLinks = [
  { to: "/student/dashboard", label: "Dashboard", icon: LayoutDashboard },
  { to: "/student/browse", label: "Browse Curricula", icon: Library },
  { to: "/student/history", label: "History", icon: Clock },
  { to: "/student/profile", label: "Profile", icon: User },
];

export default function Sidebar({ role }) {
  const links = role === "teacher" ? teacherLinks : studentLinks;
  const menuLabel = role === "teacher" ? "FACULTY MENU " : "STUDENT MENU ";

  return (
    <aside className="w-60 shrink-0 border-r border-slate-200 bg-white">
      <div className="border-b border-slate-100 px-4 py-4">
        <p className="text-xs font-bold tracking-wider text-slate-400">{menuLabel}</p>
      </div>
      <nav className="space-y-0.5 p-3">
```

```
        {links.map(({ to, label, icon: Icon }) => (
          <NavLink
            key={to}
            to={to}
            className={({ isActive }) =>
              `flex items-center gap-3 rounded-lg px-3 py-2.5 text-sm font-medium transition ${
                isActive
                  ? "sidebar-active pl-[9px]"
                  : "text-slate-600 hover:bg-slate-50 hover:text-slate-800"
              }`
            }
          >
            <Icon className="h-5 w-5 shrink-0" aria-hidden="true" />
            {label}
          </NavLink>
        ))}
      </nav>
    </aside>
  );
}

```

DashboardLayout.jsx

genai-curriculum-platform > client > src > components > shared > DashboardLayout.jsx > ...

```jsx
import { Outlet } from "react-router-dom";
import Navbar from "./Navbar";
import Sidebar from "./Sidebar";
import Chatbot from "./Chatbot";
import { useAuth } from "../../context/AuthContext";

export default function DashboardLayout() {
  const { userProfile } = useAuth();

  return (
    <div className="flex h-screen flex-col">
      <Navbar />
      <div className="flex flex-1 overflow-hidden">
        <Sidebar role={userProfile?.role} />
        <main className="flex-1 overflow-y-auto bg-[#F8F9FC] p-6">
          <Outlet />
        </main>
      </div>
      <Chatbot />
    </div>
  );
}

```

FacultyDashboard.jsx

Features:

- Curriculum Statistics
- Analytics Overview
- Recent Curricula
- Student Monitoring

Files:

src/components/teacher/

client/src/components/teacher/GenerateCurriculum.jsx

```jsx
      <div
        className={`rounded-lg px-4 py-3 text-sm ${
          toast.type === "error" ? "bg-red-50 text-red-600" : "bg-emerald-50 text-emerald-600"
        }`}
      >
        {toast.msg}
      </div>
    )}

    <div className="grid gap-6 lg:grid-cols-2">
      <form
        onSubmit={handleGenerate}
        className="rounded-xl border border-slate-100 bg-white p-6 shadow-sm"
      >
        <h2 className="mb-4 font-semibold text-slate-800">Syllabus Parameters 📝</h2>
        <div className="space-y-4">
          <div>
            <label className="mb-1 block text-sm font-medium text-slate-700">
              Degree Type
            </label>
            <select
              name="level"
              value={form.level}
              onChange={handleChange}
              className="w-full rounded-lg border border-slate-200 px-4 py-2.5 text-sm focus:border-brand focus:outline-none focus:
            >
              {DEGREE_OPTIONS.map(({ value, label }) => (
                <option key={value} value={value}>
                  {label}
                </option>
              ))}
            </select>
          </div>
          <div>
            <label className="mb-1 block text-sm font-medium text-slate-700">
              Course Title / Specialization
            </label>
            <input
              name="skill"
              required
              value={form.skill}
              onChange={handleChange}
              placeholder="e.g. Artificial Intelligence & Data Science"
              className="w-full rounded-lg border border-slate-200 px-4 py-2.5 text-sm focus:border-brand focus:outline-none focus:
            />
```

```
            />
          </div>
          <div>
            <label className="mb-1 block text-sm font-medium text-slate-700">
              Program Duration
            </label>
            <select
              name="semesters"
              value={form.semesters}
              onChange={handleChange}
              className="w-full rounded-lg border border-slate-200 px-4 py-2.5 text-sm focus:border-brand focus:outline-none focus:
            >
              {DURATION_OPTIONS.map(({ value, label }) => (
                <option key={value} value={value}>
                  {label}
                </option>
              ))}
            </select>
          </div>
          <div>
            <label className="mb-1 block text-sm font-medium text-slate-700">
              Industrial Skill Tags / Focus Areas
            </label>
            <input
              name="industryFocus"
              value={form.industryFocus}
              onChange={handleChange}
              placeholder="python, database systems, neural networks"
              className="w-full rounded-lg border border-slate-200 px-4 py-2.5 text-sm focus:border-brand focus:outline-none focus:
            />
```

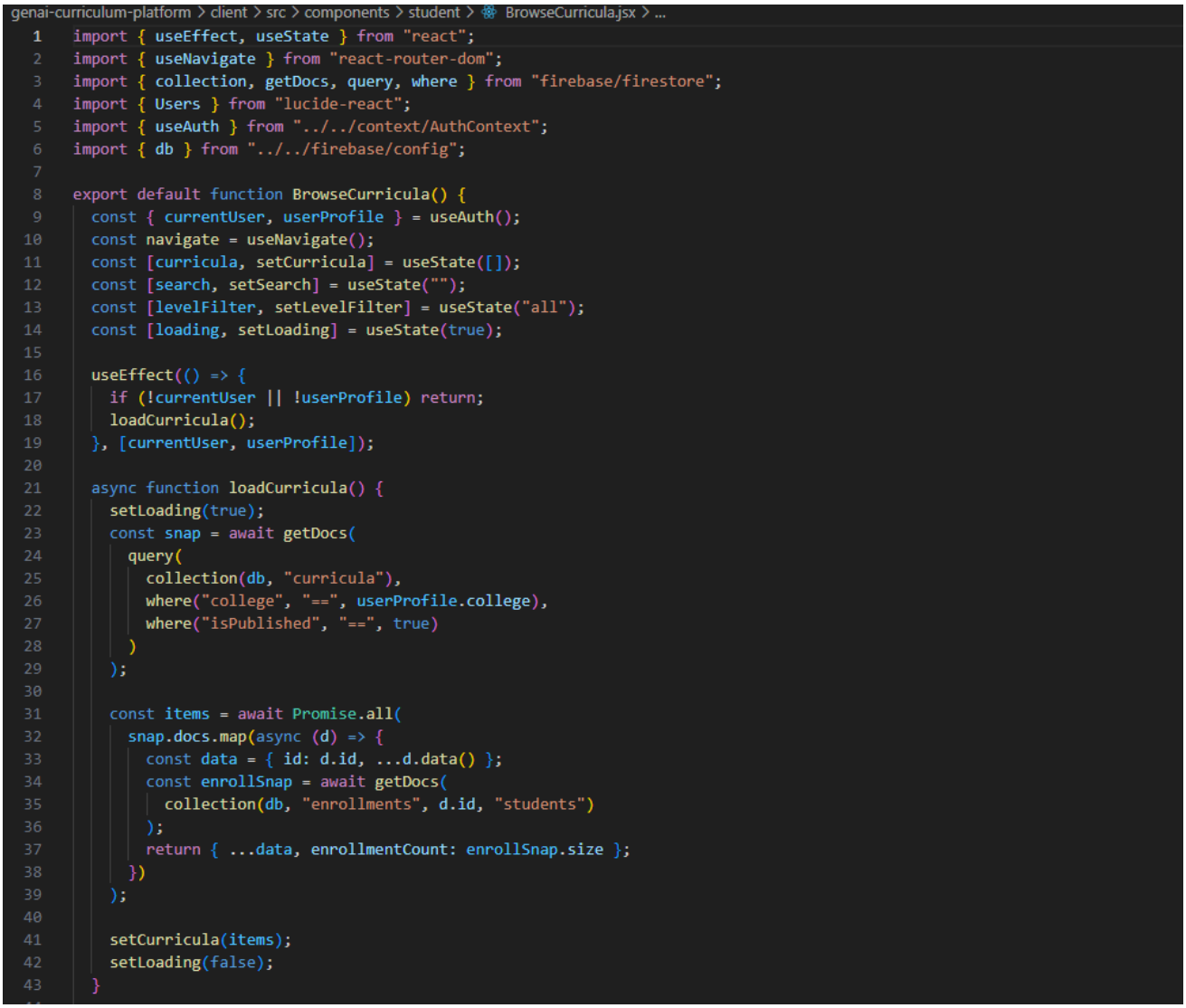
```
            <p className="mt-1 text-xs text-slate-400">Separate multiple skills with commas.</p>
          </div>
          <div>
            <label className="mb-1 block text-sm font-medium text-slate-700">
              Weekly Hours (optional)
            </label>
            <input
              name="weeklyHours"
              value={form.weeklyHours}
              onChange={handleChange}
              placeholder="20-25"
              className="w-full rounded-lg border border-slate-200 px-4 py-2.5 text-sm focus:border-brand focus:outline-none focus:
            />
          </div>
        </div>
        <button
          type="submit"
          disabled={loading}
          className="mt-5 flex w-full items-center justify-center gap-2 rounded-lg bg-brand py-3 text-sm font-semibold text-white t
        >
          {loading ? "Generating…" : <>Generate Syllabus Map 🪄</>}
        </button>
      </form>

      <div className="rounded-xl border border-slate-100 bg-white p-6 shadow-sm">
        {loading ? (
          <div className="flex h-full min-h-[320px] flex-col items-center justify-center gap-3">
            <div className="h-10 w-10 animate-spin rounded-full border-4 border-brand border-t-transparent" />
            <span className="text-sm text-slate-500">Generating curriculum with AI…</span>
          </div>
```

Student Components

Files:

src/components/student/

client/src/components/student/BrowseCurricula.jsx




```jsx
import { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import { collection, getDocs, query, where } from "firebase/firestore";
import { Users } from "lucide-react";
import { useAuth } from "../../context/AuthContext";
import { db } from "../../firebase/config";

export default function BrowseCurricula() {
  const { currentUser, userProfile } = useAuth();
  const navigate = useNavigate();
  const [curricula, setCurricula] = useState([]);
  const [search, setSearch] = useState("");
  const [levelFilter, setLevelFilter] = useState("all");
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    if (!currentUser || !userProfile) return;
    loadCurricula();
  }, [currentUser, userProfile]);

  async function loadCurricula() {
    setLoading(true);
    const snap = await getDocs(
      query(
        collection(db, "curricula"),
        where("college", "==", userProfile.college),
        where("isPublished", "==", true)
      )
    );

    const items = await Promise.all(
      snap.docs.map(async (d) => {
        const data = { id: d.id, ...d.data() };
        const enrollSnap = await getDocs(
          collection(db, "enrollments", d.id, "students")
        );
        return { ...data, enrollmentCount: enrollSnap.size };
      })
    );

    setCurricula(items);
    setLoading(false);
  }
```

```
44
45    const filtered = curricula.filter((c) => {
46      const matchesSearch =
47        !search ||
48        c.title.toLowerCase().includes(search.toLowerCase()) ||
49        c.skill.toLowerCase().includes(search.toLowerCase());
50      const matchesLevel = levelFilter === "all" || c.level === levelFilter;
51      return matchesSearch && matchesLevel;
52    });
53
54    if (loading) {
55      return (
56        <div className="flex justify-center py-20">
57          <div className="h-8 w-8 animate-spin rounded-full border-4 border-brand border-t-transparent" />
58        </div>
59      );
60    }
61
62    return (
63      <div className="space-y-6">
64        <h1 className="text-2xl font-bold text-slate-800">Browse Curricula</h1>
65
66        <div className="flex flex-wrap gap-3">
67          <input
68            value={search}
69            onChange={(e) => setSearch(e.target.value)}
70            placeholder="Search by title or skill…"
71            className="flex-1 rounded-lg border border-slate-200 px-4 py-2.5 text-sm focus:border-brand focus:outline-none min-w-[200p
72          />
73          <select
74            value={levelFilter}
75            onChange={(e) => setLevelFilter(e.target.value)}
76            className="rounded-lg border border-slate-200 px-4 py-2.5 text-sm focus:border-brand focus:outline-none"
77          >
78            <option value="all">All Levels</option>
79            {["Diploma", "BTech", "Masters", "Certification"].map((l) => (
80              <option key={l} value={l}>{l}</option>
81            ))}
82          </select>
83        </div>
```

```
85        {filtered.length === 0 ? (
86          <p className="py-12 text-center text-slate-400">No curricula found.</p>
87        ) : (
88          <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-3">
89            {filtered.map((c) => (
90              <button
91                key={c.id}
92                onClick={() => navigate(`/student/curriculum/${c.id}`)}
93                className="rounded-xl border border-slate-100 bg-white p-5 text-left shadow-sm transition hover:shadow-md"
94              >
95                <h3 className="font-semibold text-slate-800">{c.title}</h3>
96                <p className="mt-1 text-sm text-slate-500">{c.skill} · {c.level}</p>
97                <p className="mt-1 text-xs text-slate-400">{c.semesters} semesters</p>
98                <div className="mt-3 flex items-center justify-between">
99                  <span className="text-xs text-slate-500">By {c.teacherName}</span>
100                 <span className="flex items-center gap-1 text-xs text-brand">
101                   <Users className="h-3 w-3" />
102                   {c.enrollmentCount} enrolled
103                 </span>
104               </div>
105             </button>
106           ))}
107         </div>
108       )}
109     </div>
110   );
111 }
112
```

MILESTONE 7: Integration & Testing

This phase integrates all modules and validates system functionality.

Activity 7.1: End-to-End Integration

Modules Integrated:

- React Frontend
- Express Backend
- Firebase
- Firestore
- Groq AI

Activity 7.2: Functional Testing

Functional testing was conducted to verify that all major modules of the EduCanvas platform operate according to the specified requirements. Each feature was tested independently and then validated as part of the integrated system.

Test Case 1: User Login Functionality

Objective

To verify that registered users can successfully log into the system using valid credentials. **Test**

Procedure

1. Open the Login page.
2. Enter a valid email address and password.
3. Click the **Sign In** button.

Expected Result

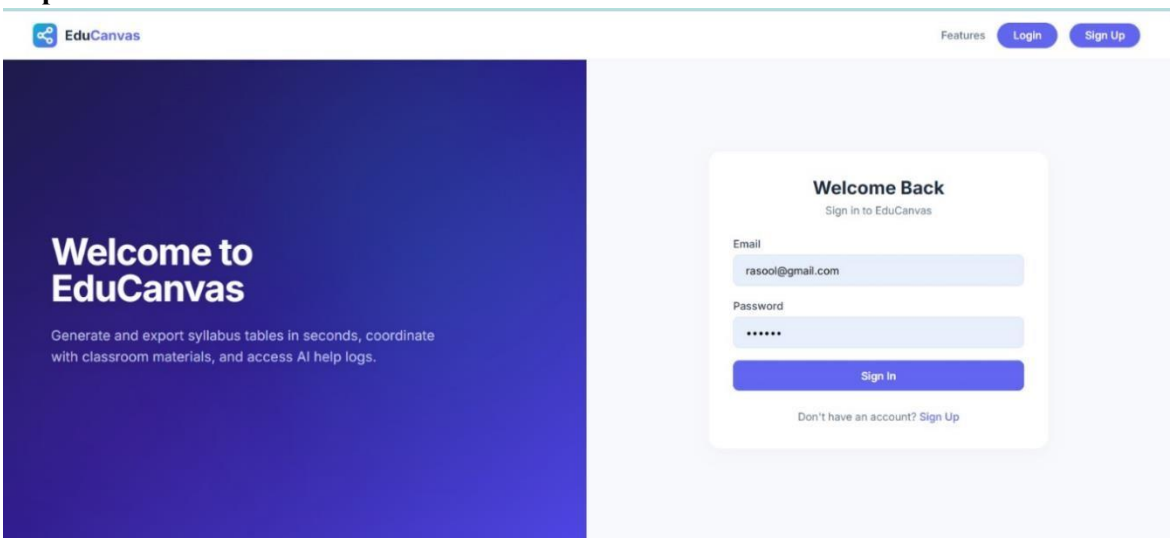
The user should be authenticated and redirected to the appropriate dashboard.

Actual Result

The user successfully logged into the platform and was redirected to the dashboard.

Status PASS

Output Screenshot:



Test Case 2: Multi-Step User Registration

Objective

To verify that new users can create an account through the three-step registration wizard.

Test Procedure

1. Select account type.
2. Enter personal information.
3. Provide institution details.
4. Complete registration.

Expected Result

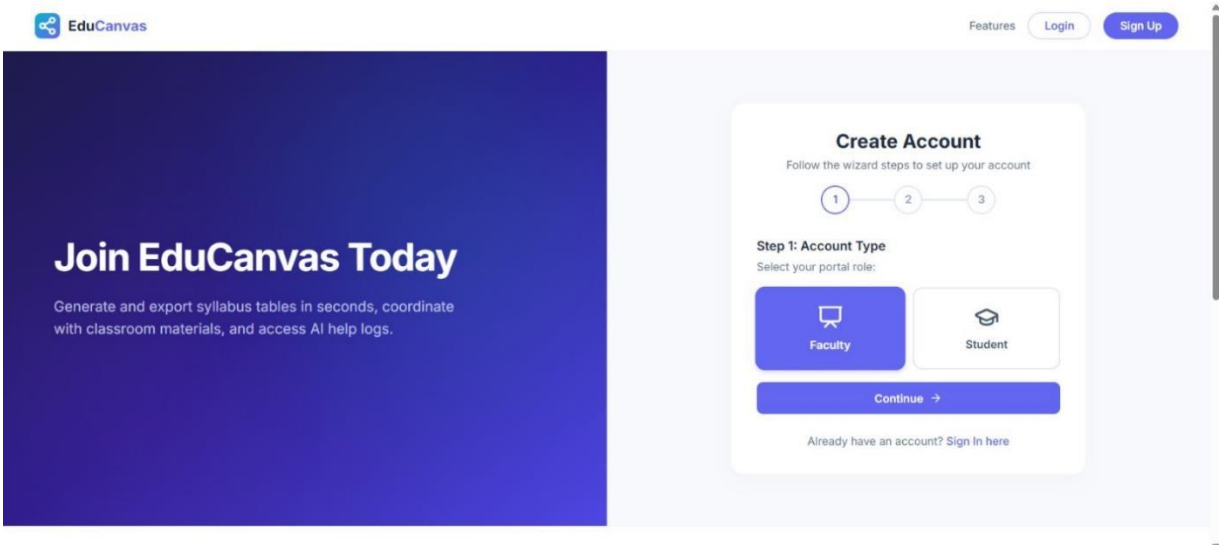
A new account should be created and stored in Firebase Authentication and Firestore. **Actual Result**

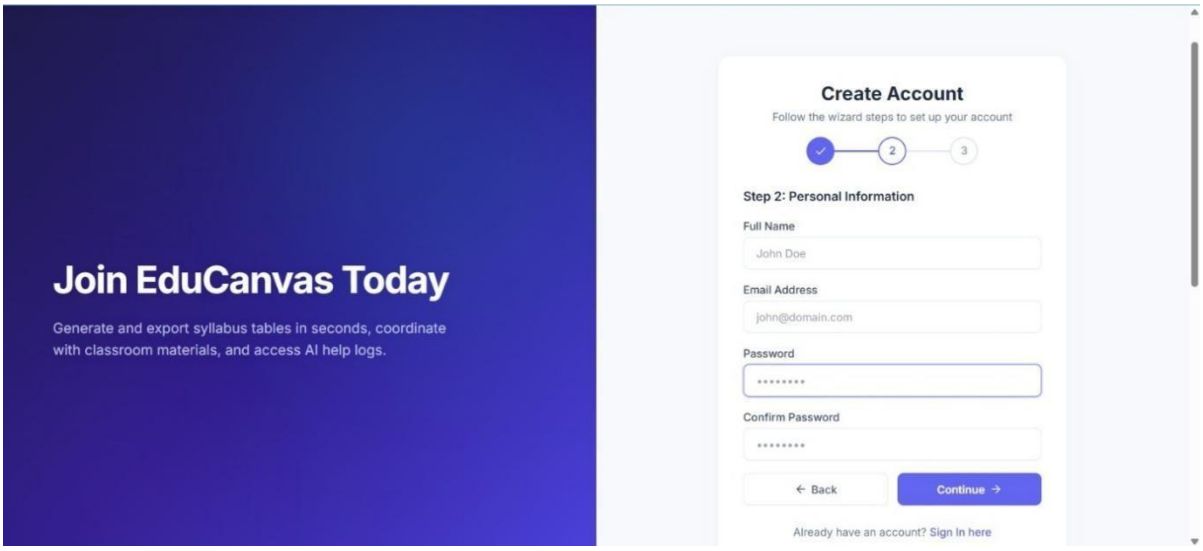
Account registration completed successfully.

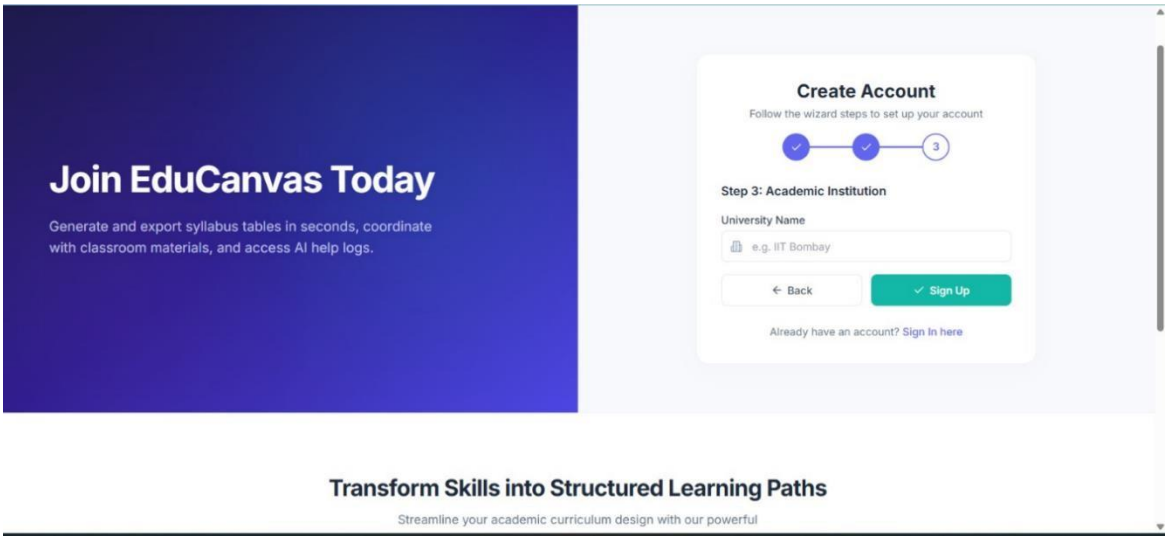
Status

PASS

Output Screenshots:







Test Case 3: Student Dashboard Access

Objective

To verify that students can access their personalized dashboard after login.

Test Procedure

1. Login as a student.
2. Navigate to Dashboard.

Expected Result

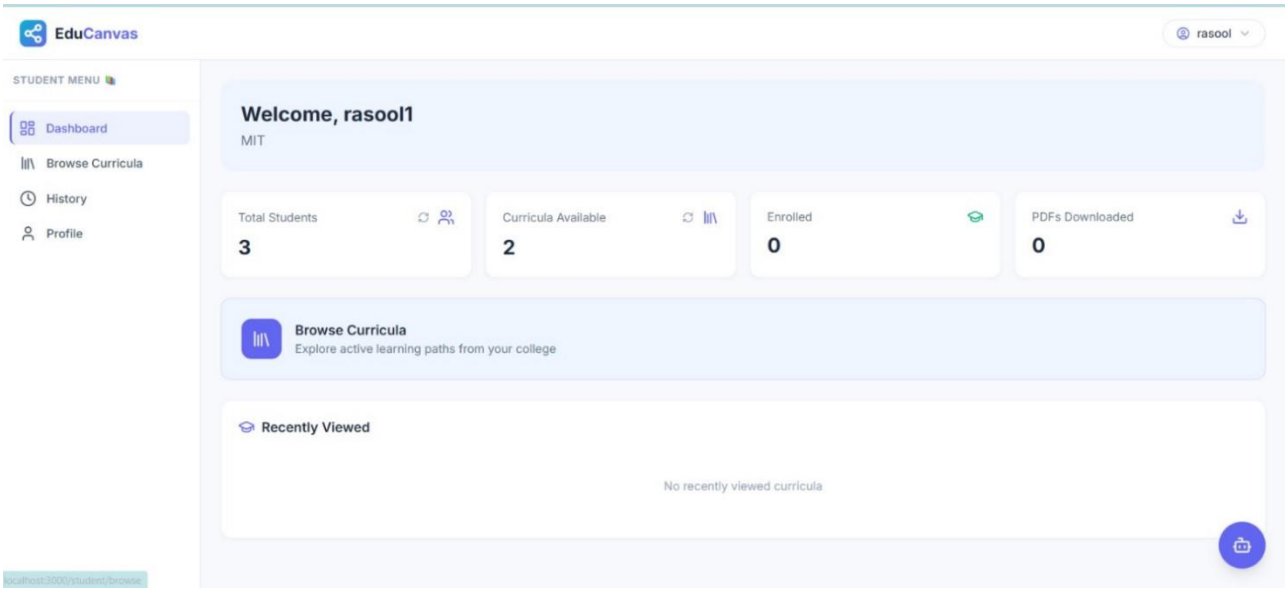
Dashboard should display curriculum statistics, enrollment information, and recent activity. **Actual**

Result

Student dashboard loaded successfully with all relevant information.

Status PASS

Output Screenshot:



Test Case 4: Curriculum Browsing

Objective

To verify that students can browse available curricula.

Test Procedure

1. Login as a student.
2. Open Browse Curricula page.
3. View available learning plans.

Expected Result

All published curricula should be displayed.

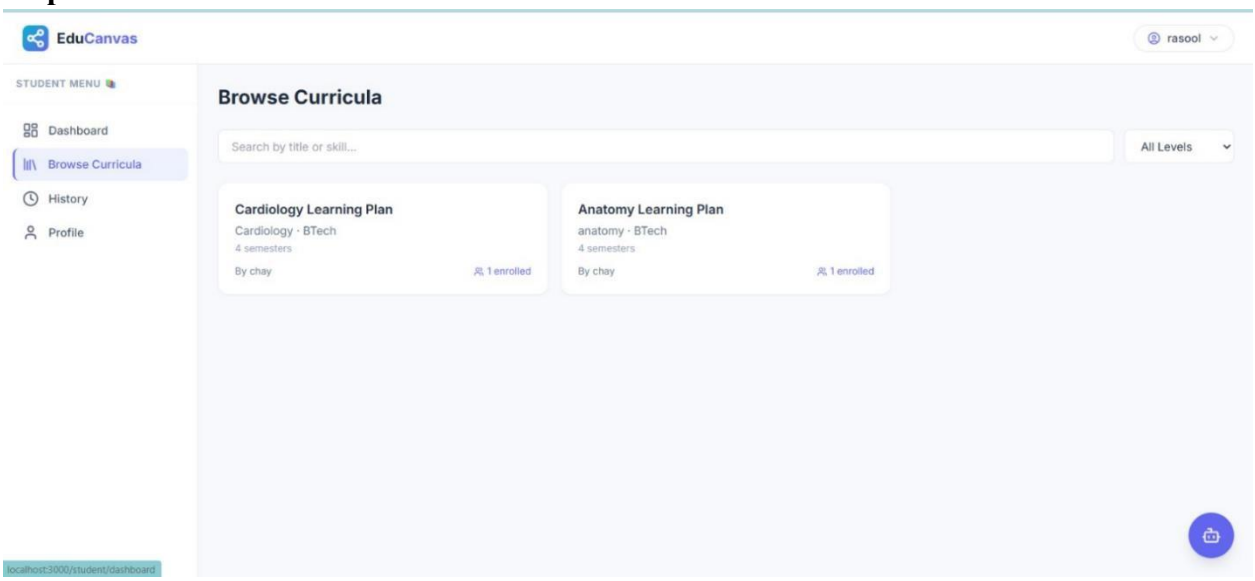
Actual Result

Curriculum cards were displayed correctly with enrollment details.

Status

PASS

Output Screenshot:



Test Case 5: Student Activity History Tracking

Objective

To verify that curriculum activities are recorded in the history module.

Test Procedure

1. View curriculum.
2. Download curriculum PDF.
3. Enroll in curriculum.
4. Open History page.

Expected Result

All actions should be recorded and displayed.

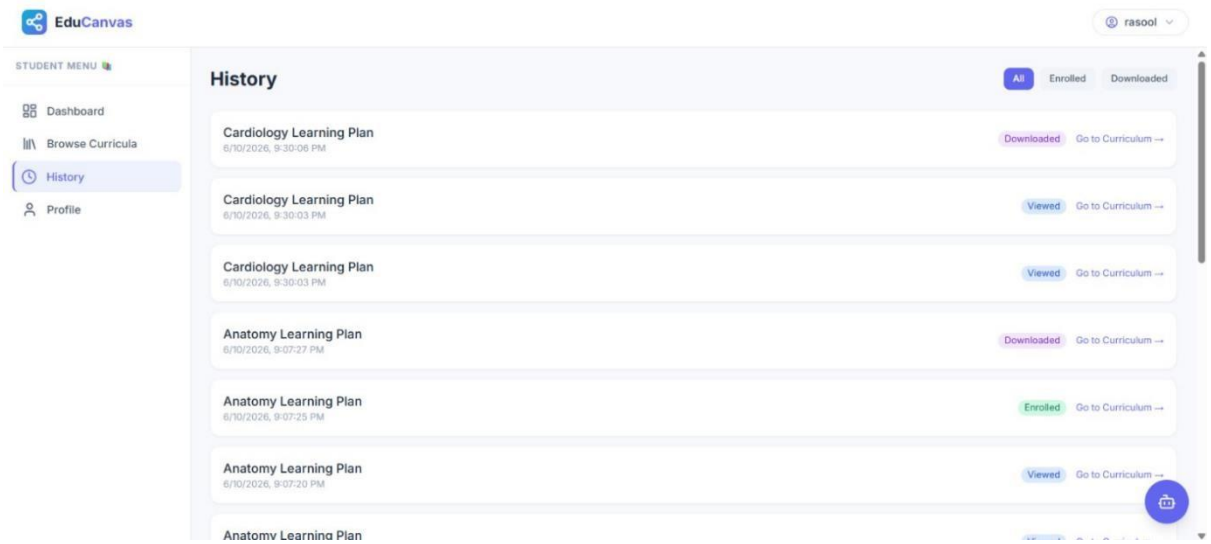
Actual Result

Viewed, enrolled, and downloaded activities were successfully tracked.

Status

PASS

Output Screenshot:



Test Case 6: Faculty Curriculum Management

Objective

To verify that faculty members can manage generated curricula.

Test Procedure

1. Login as faculty.
2. Open My Curricula.
3. View student progress.
4. Edit, export, or delete curriculum.

Expected Result

Faculty should be able to manage curriculum records and monitor student progress.

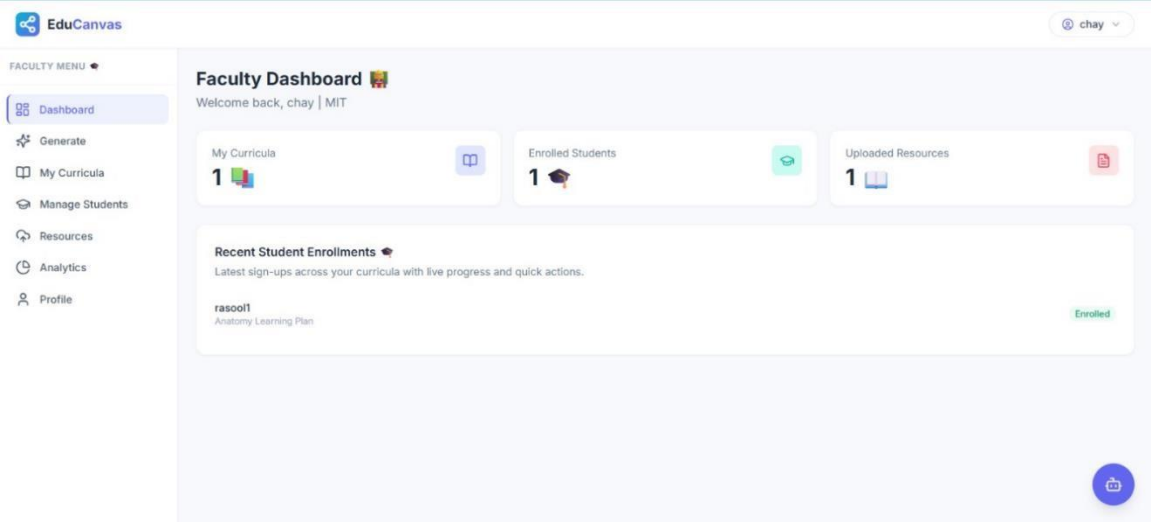
Actual Result

Curriculum management operations executed successfully.

Status

PASS

Output Screenshot:



Test Case 7: Student Progress Monitoring

Objective

To verify that curriculum progress tracking works correctly.

Test Procedure

1. Student enrolls in a curriculum.
2. Student accesses learning content.
3. Faculty reviews progress.

Expected Result

Progress percentage and completion statistics should be displayed.

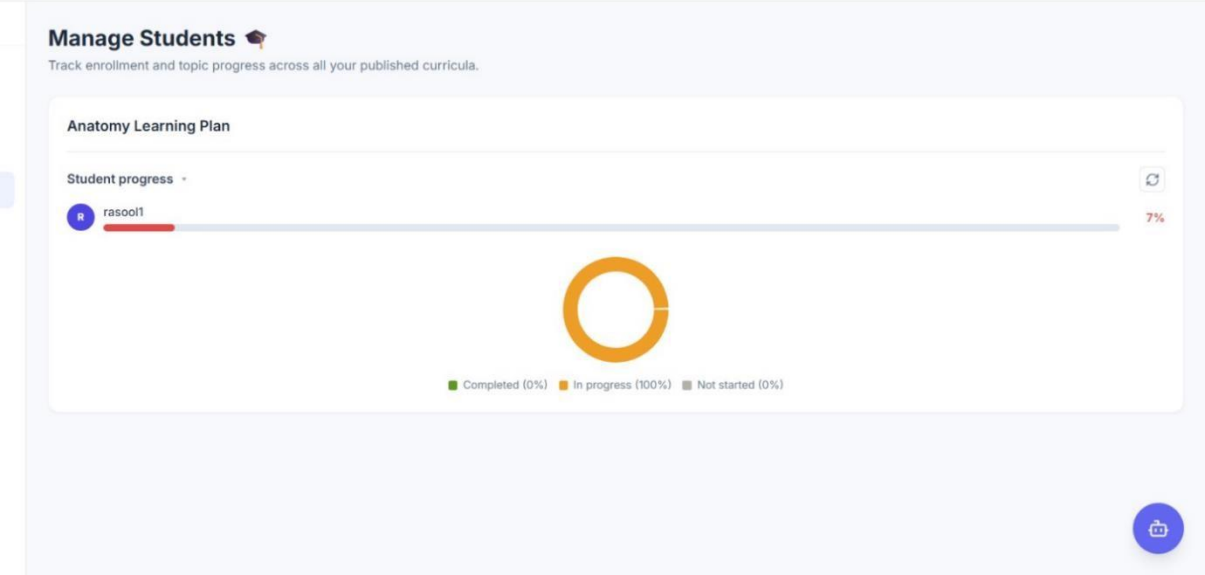
Actual Result

Progress indicators and charts displayed correctly.

Status

PASS

Output Screenshot:



Test Case 8: Analytics Dashboard

Objective

To verify that analytical reports and enrollment statistics are generated correctly.

Test Procedure

1. Login as faculty.
2. Open Analytics page.
3. Review enrollment charts and activity metrics.

Expected Result

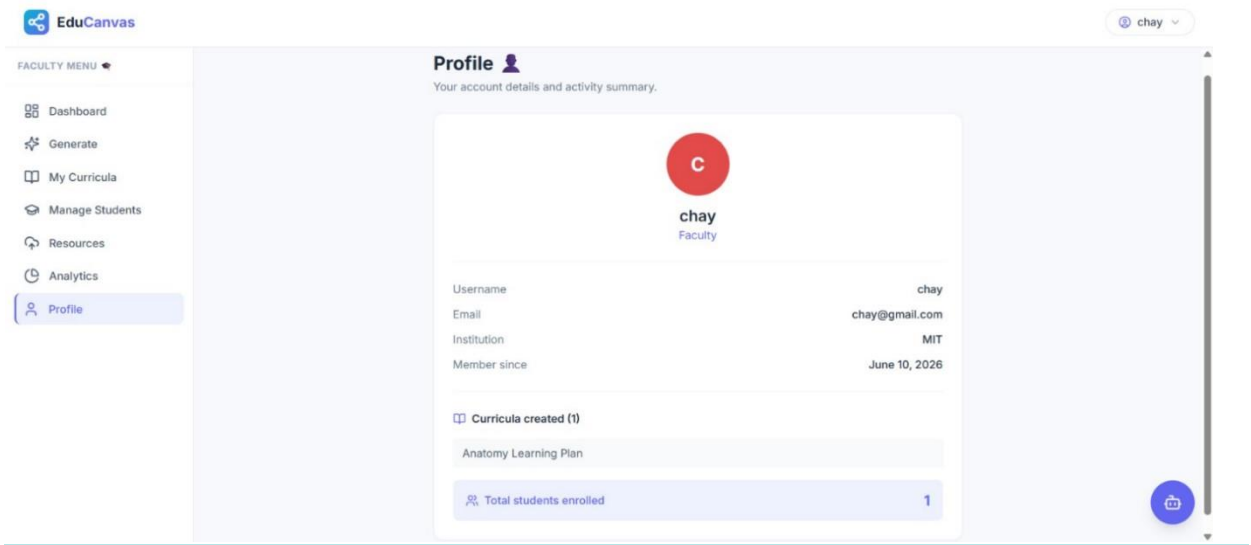
Enrollment statistics, curriculum popularity, and activity reports should be displayed. **Actual Result**

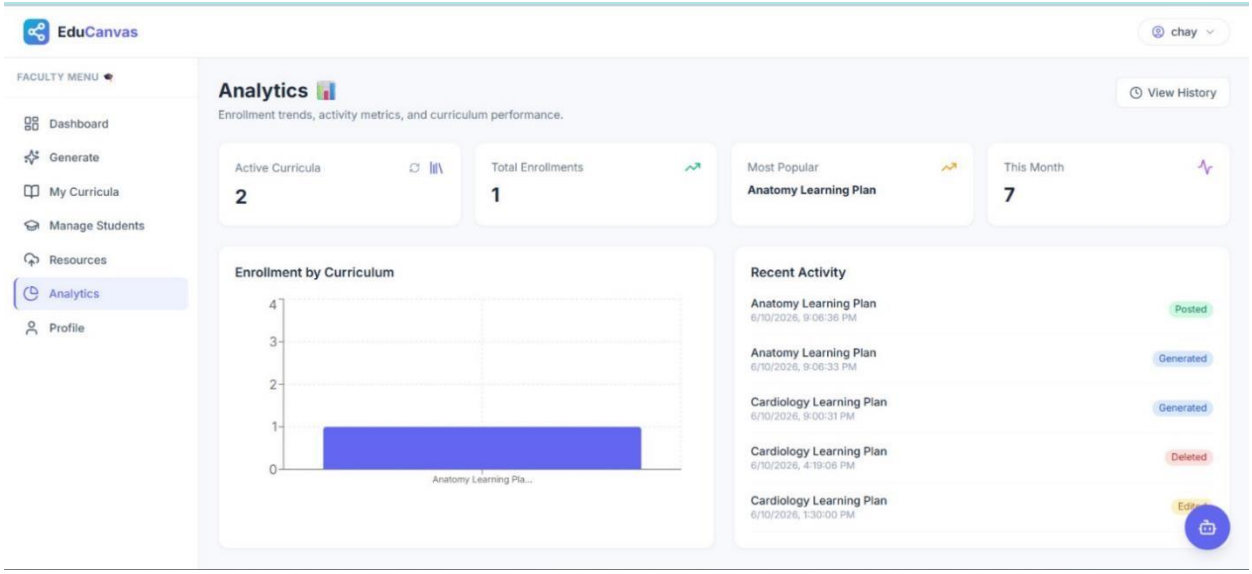
Analytics dashboard generated charts and reports successfully.

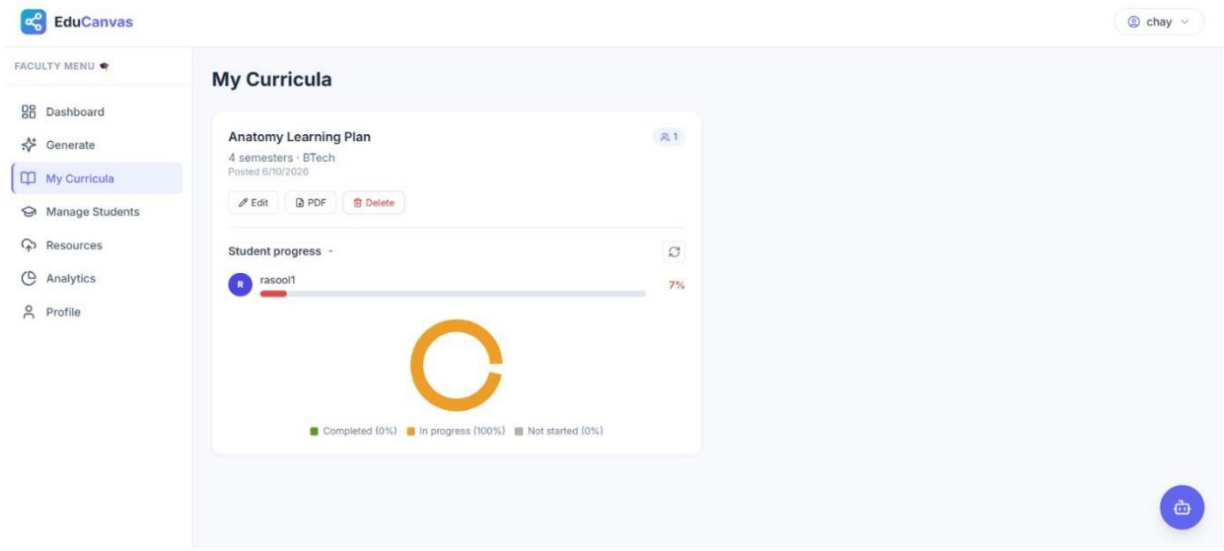
Status

PASS

Output Screenshot:


EduCanvas
chay
FACULTY MENU
Dashboard
Generate
My Curricula
Manage Students
Resources
Analytics
Profile
Profile
Your account details and activity summary.
C
chay
Faculty
Username
chay
Email
chay@gmail.com
Institution
MIT
Member since
June 10, 2026
Curricula created (1)
Anatomy Learning Plan
Total students enrolled
1


EduCanvas
chay
FACULTY MENU
Dashboard
Generate
My Curricula
Manage Students
Resources
Analytics
Profile
Analytics
Enrollment trends, activity metrics, and curriculum performance.
View History
Active Curricula
2
Total Enrollments
1
Most Popular
Anatomy Learning Plan
This Month
7
Enrollment by Curriculum
4
3
2
1
0
Anatomy Learning Pla...
Recent Activity
Anatomy Learning Plan
6/10/2026, 9:06:36 PM
Posted
Anatomy Learning Plan
6/10/2026, 9:06:33 PM
Generated
Cardiology Learning Plan
6/10/2026, 9:00:31 PM
Generated
Cardiology Learning Plan
6/10/2026, 4:19:06 PM
Deleted
Cardiology Learning Plan
6/10/2026, 1:30:00 PM



Functional Testing Summary

Module	Result
User Authentication	PASS
User Registration	PASS
Student Dashboard	PASS
Curriculum Browsing	PASS
History Tracking	PASS
Curriculum Management	PASS
Progress Monitoring	PASS
Analytics Dashboard	PASS

Overall Result

All functional requirements of the EduCanvas platform were tested successfully. The system performed as expected without critical failures, confirming that the application is ready for deployment and user acceptance testing.

Activity 7.3: Performance Testing

Validated:

- API Response Time
- Firestore Performance
- Dashboard Loading
- Curriculum Generation Speed

MILESTONE 8: Deployment & Documentation

This phase deploys the EduCanvas platform and prepares project documentation.

Activity 8.1: Application Deployment

Deployment Platform:

- Vercel

Deployment Steps:

1. Push project to GitHub
2. Connect repository to Vercel
3. Configure Environment Variables
4. Deploy Application

Activity 8.2: Documentation

Prepared Documents:

- Technical Documentation
- User Manual
- Installation Guide
- Deployment Guide
- API Documentation

Conclusion

EduCanvas successfully demonstrates the integration of Artificial Intelligence with modern web technologies to streamline curriculum design and academic management. The platform automates curriculum generation, enhances educational planning, improves student accessibility, and provides secure institution-specific academic environments.

By combining React, Node.js, Firebase, Firestore, and Groq AI technologies, EduCanvas delivers a scalable, intelligent, and user-friendly solution capable of supporting modern educational institutions. The project highlights the potential of AI-driven educational systems and establishes a foundation for future advancements in curriculum management and personalized learning.